Assignment- 17.2

Name : vamshi krishna

Htno: 2303A51452

Batch : 22

Task Description – 1(Student Academic Data Cleaning)

- **Task:**

Use AI assistance to generate a Python script that cleans and prepares a student academic dataset for analysis.

Instructions:

- **Handle missing values in marks, attendance, and department.**
- **Remove duplicate student records based on student ID.**
- **Standardize text fields such as department names.**
- **Encode categorical variables like gender and department.**

Sample Code:

```
import pandas as pd
data = pd.read_csv("students.csv")
print(data.head())
```

Expected Output:

- **A cleaned and encoded Pandas DataFrame suitable for academic performance analysis.**

Prompt:

Generate Python code using pandas to clean a student dataset. Handle missing values, remove duplicates based on student ID, standardize department names, and encode categorical variables.

Code:

task1.py ● students.csv BLACKBOX

task1.py > ...

```python
#Generate Python code using pandas to clean a student dataset.
#Handle missing values, remove duplicates based on student ID,
#standardize department names, and encode categorical variables.
import pandas as pd
# Load data
data = pd.read_csv("students.csv")
# Handle missing values (FIXED)
data['marks'] = data['marks'].fillna(data['marks'].mean())
data['attendance'] = data['attendance'].fillna(data['attendance'].median
data['department'] = data['department'].fillna("Unknown")
# Remove duplicates
data = data.drop_duplicates(subset='student_id')
# Standardize text
data['department'] = data['department'].str.lower().str.strip()
# Encode categorical variables
data['gender'] = data['gender'].map({'Male': 0, 'Female': 1})
data = pd.get_dummies(data, columns=['department'])
print(data)
```

PROBLEMS OUTPUT TERMINAL ··· powershell

```
PS C:\Users\dell\OneDrive\Desktop\AI\Lab17> python task1.py
   student_id     name  ...  department_ece  department_unknown
0           1    Alice  ...           False               False
1           2      Bob  ...            True               False
2           3  Charlie  ...           False               False
3           4    David  ...           False                True

[4 rows x 8 columns]
PS C:\Users\dell\OneDrive\Desktop\AI\Lab17>
```

students.csv > data

```csv
student_id,name,marks,attendance,department,gender
1,Alice,85,90,CSE,Female
2,Bob,,80,ECE,Male
3,Charlie,78,,CSE,Male
4,David,92,95,,Male
1,Alice,85,90,CSE,Female
```

Explanation:

The dataset was loaded using pandas. Missing values in marks, attendance, and department columns were handled using mean, median, and default values respectively. Duplicate student records were removed based on student ID. The department names were standardized by converting them into lowercase. Categorical variables such as gender and department were encoded into numerical format using mapping and one-hot encoding techniques.

Task Description – 2 (Financial Transaction Data Preprocessing)

- **Task:**

Preprocess financial transaction data using AI-generated Python code.

Instructions:

- **Convert transaction date columns to datetime format.**
- **Create derived features such as transaction month and year.**
- **Normalize the transaction amount column.**
- **Identify and handle extreme transaction values.**

Sample Code:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
transactions = pd.read_csv("transactions.csv")
print(transactions.info())
```

Expected Output:

- **A transformed dataset with normalized values and newly generated time-based features.**

Prompt:

Generate Python code to preprocess financial transaction data. Convert date columns, extract month/year, normalize amount, and detect extreme values.

Code:

```
py    students.csv    transactions.csv    task2.py    BLACKBOX
task2.py > ...
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
# Load data
transactions = pd.read_csv("transactions.csv")
# Convert date column
transactions['date'] = pd.to_datetime(transactions['date'])
# Create new features
transactions['month'] = transactions['date'].dt.month
transactions['year'] = transactions['date'].dt.year
# Normalize amount
scaler = MinMaxScaler()
transactions['amount_normalized'] = scaler.fit_transform(transact
# Remove outliers using IQR
Q1 = transactions['amount'].quantile(0.25)
Q3 = transactions['amount'].quantile(0.75)
IQR = Q3 - Q1
transactions = transactions[
    (transactions['amount'] >= Q1 - 1.5 * IQR) &
    (transactions['amount'] <= Q3 + 1.5 * IQR)
]
# Display output
```

```
PROBLEMS    OUTPUT    TERMINAL    ...    powershell

PS C:\Users\dell\OneDrive\Desktop\AI\Lab17> python task2.py
   transaction_id       date  amount  month  year  amount_normalized
0               1 2023-01-15    5000      1  2023           0.047141
1               2 2023-02-20   15000      2  2023           0.147442
2               3 2023-03-10     300      3  2023           0.000000
4               5 2023-05-18     700      5  2023           0.004012
PS C:\Users\dell\OneDrive\Desktop\AI\Lab17>
```

task1.py | students.csv | transaction

transactions.csv > data

```csv
transaction_id,date,amount
1,2023-01-15,5000
2,2023-02-20,15000
3,2023-03-10,300
4,2023-04-05,100000
5,2023-05-18,700
```

Explanation:
The transaction dataset was loaded and the date column was converted into datetime format. New features such as month and year were extracted from the date column. The transaction amount was normalized using MinMaxScaler to bring all values into a standard range. Outliers were detected and removed using the Interquartile Range (IQR) method to improve data quality.

Task 3 – (Environmental Sensor Data Preparation)

- **Task:**

Clean and preprocess environmental sensor data using AI-assisted scripts.

Instructions:

- **Handle missing values in temperature, humidity, and air_quality_index.**
- **Scale numerical features using standard scaling.**
- **Encode categorical fields such as sensor_status.**
- **Split the dataset into training and testing subsets.**

Sample Code:

import pandas as pd

from sklearn.model_selection import train_test_split
data = pd.read_csv("sensor_data.csv")
Expected Output:

- **A fully preprocessed dataset ready for predictive modeling tasks.**

Prompt:

Generate Python code to clean sensor data.
Handle missing values, scale numerical features,
encode categorical fields, and split dataset**.**

Code:

task3.py

```python
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
# Load data
data = pd.read_csv("sensor_data.csv")
# Handle missing values
data['temperature'] = data['temperature'].fillna(data['temperatur
data['humidity'] = data['humidity'].fillna(data['humidity'].mean()
data['air_quality_index'] = data['air_quality_index'].fillna(data
# Encode categorical variable
data['sensor_status'] = data['sensor_status'].map({'active': 1, 'inact
# Scale numerical features
scaler = StandardScaler()
num_cols = ['temperature', 'humidity', 'air_quality_index']
data[num_cols] = scaler.fit_transform(data[num_cols])
# Split dataset
train_data, test_data = train_test_split(data, test_size=0.2, ran
```

PROBLEMS OUTPUT TERMINAL powershell

```
PS C:\Users\dell\OneDrive\Desktop\AI\Lab17> python task3.py
TRAIN DATA:
    temperature  humidity  air_quality_index  sensor_status
4      1.621266       1.5                1.5              0
2      0.000000      -1.5               -1.5              0
0     -0.540422      -0.5               -0.5              1
3     -1.405098       0.5                0.0              1

TEST DATA:
    temperature  humidity  air_quality_index  sensor_status
1      0.324253       0.0                0.5              1
PS C:\Users\dell\OneDrive\Desktop\AI\Lab17>
```

Explanation:
Missing values in temperature, humidity, and air quality index were handled using mean imputation. The categorical variable sensor_status was encoded into numerical values. Numerical features were standardized using StandardScaler to ensure a uniform scale. The dataset was then split into training and testing sets using train_test_split for further machine learning tasks.

Task 4 – (Real-Time Application: Smart City Traffic Data Cleaning)

- **Scenario:**

A smart city system collects traffic data from multiple sensors, resulting in noisy and inconsistent records.

Requirements:

- **Standardize road and location names.**
- **Fill missing traffic density values using appropriate statistical methods.**
- **Remove duplicate sensor readings.**
- **Normalize speed and vehicle count metrics.**
- **Generate a brief summary comparing dataset quality before and after cleaning**

Sample Code:

```
import pandas as pd
traffic = pd.read_csv("traffic_data.csv")
print(traffic.describe())
```

Expected Output:

- **A cleaned and normalized traffic dataset with an accompanying data-quality improvement summary.**

Prompt:
Generate Python code to clean sensor data.
Handle missing values, scale numerical features,
encode categorical fields, and split dataset.
Code:

task4.py > ...

```python
#Standardize road and location names, handle missing traffic dens
#remove duplicate records, normalize speed and vehicle count,
#and compare dataset quality before and after cleaning.
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
# Load data
traffic = pd.read_csv("traffic_data.csv")
# Before cleaning summary
print("===== BEFORE CLEANING =====")
print(traffic.describe(include='all'))
# Standardize text fields
traffic['road'] = traffic['road'].str.lower().str.strip()
traffic['location'] = traffic['location'].str.lower().str.strip()
# Handle missing values
traffic['traffic_density'] = traffic['traffic_density'].fillna(tr
# Remove duplicates
traffic = traffic.drop_duplicates()
# Normalize numerical features
scaler = MinMaxScaler()
traffic[['speed', 'vehicle_count']] = scaler.fit_transform(
    traffic[['speed', 'vehicle_count']]
)
# After cleaning summary
print("\n===== AFTER CLEANING =====")
print(traffic.describe(include='all'))
```

Output:

```
PS C:\Users\dell\OneDrive\Desktop\AI\Lab17> python task4.py
===== BEFORE CLEANING =====
           road   location  traffic_density       speed  vehicle_count
count         6          6         4.000000    6.000000       6.000000
unique        5          5              NaN         NaN            NaN
top     Highway  Outskirts              NaN         NaN            NaN
freq          2          2              NaN         NaN            NaN
mean        NaN        NaN        60.000000   66.666667     248.333333
std         NaN        NaN        24.494897   26.770631     118.729384
min         NaN        NaN        30.000000   40.000000     150.000000
25%         NaN        NaN        45.000000   47.500000     165.000000
50%         NaN        NaN        65.000000   57.500000     190.000000
75%         NaN        NaN        80.000000   90.000000     350.000000
max         NaN        NaN        80.000000  100.000000     400.000000

===== AFTER CLEANING =====
             road     location  traffic_density     speed  vehicle_count
count           5            5         5.000000  5.000000        5.00000
unique          3            3              NaN       NaN            NaN
top     main road  city center              NaN       NaN            NaN
freq            2            2              NaN       NaN            NaN
mean          NaN          NaN        56.000000  0.333333        0.27200
std           NaN          NaN        18.165902  0.395285        0.41415
min           NaN          NaN        30.000000  0.000000        0.00000
25%           NaN          NaN        50.000000  0.083333        0.04000
unique          3            3              NaN       NaN            NaN
top     main road  city center              NaN       NaN            NaN
freq            2            2              NaN       NaN            NaN
mean          NaN          NaN        56.000000  0.333333        0.27200
```

Explanation:
Missing values in temperature, humidity, and air quality index were handled using mean imputation. The categorical variable sensor_status was encoded into numerical values. Numerical features were standardized using StandardScaler to ensure a uniform scale. The dataset was then split into training and testing sets using train_test_split for further machine learning tasks.

Task 5 – (Social Media Text Data Cleaning and Feature Extraction)

- **Task:**

Use AI-assisted Python scripting to clean and preprocess social media text data for sentiment analysis.

Instructions:

- **Remove duplicate posts and null text entries.**
- **Clean text by removing URLs, special characters, and extra spaces.**
- **Convert text to lowercase and perform basic tokenization.**
- **Remove stopwords and perform stemming or lemmatization.**
- **Generate basic text features such as word count and sentiment score.**

Sample Code:

```
import pandas as pd
import re
posts = pd.read_csv("social_media_posts.csv")
print(posts.head())
```

Expected Output:

A cleaned text dataset with extracted features suitable for sentiment analysis models.

Prompt:

Generate Python code to clean social media text data.
Remove duplicates and null values, clean text,
tokenize, remove stopwords, and extract features like sentiment.

Output:

task5.py > ...

```
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.sentiment import SentimentIntensityAnalyzer
# Download required resources (first time only)
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('vader_lexicon')
# Load data
posts = pd.read_csv("social_media_posts.csv")
# Remove duplicates and null values
posts = posts.drop_duplicates()
posts = posts.dropna(subset=['text'])
# Clean text function
def clean_text(text):
    text = re.sub(r"http\S+", "", text)          # remove URLs
    text = re.sub(r"[^a-zA-Z ]", "", text)       # remove special
    text = text.lower().strip()                 # lowercase
    return text
posts['clean_text'] = posts['text'].apply(clean_text)
# Tokenization and stopword removal
```

```
[nltk_data] Downloading package vader_lexicon to
[nltk_data]     C:\Users\dell\AppData\Roaming\nltk_data...
[nltk_data]   Package vader_lexicon is already up-to-date!
                        text  ... sentiment
0  I love AI! https://ai.com  ...    0.6369
1             This is bad!!!  ...   -0.5423
2      Amazing experience :)  ...    0.5859
5        AI is the future!!!  ...    0.0000
6      Worst service ever!!!  ...   -0.6249

[5 rows x 5 columns]
PS C:\Users\dell\OneDrive\Desktop\AI\Lab17>
```

Explanation:
The dataset was cleaned by removing duplicate and null entries. Text preprocessing included removing URLs, special characters, and converting text to lowercase. Tokenization was performed using simple splitting, and stopwords were removed to retain meaningful words. Additional features such as word count and sentiment score were extracted using sentiment analysis techniques.